

BUILDING & DEPLOYING JDBC APPLICATIONS ON LINUX

A Comprehensive, Step-by-Step Practical Guide for Beginners

As a Senior Java Engineer, I have built and maintained numerous production-grade enterprise applications. Whether you are moving toward advanced web frameworks (like Spring Boot or Jakarta EE) or classic Servlet/JSP architectures, understanding **Java Database Connectivity (JDBC)** is absolutely fundamental. JDBC acts as the bridge between your Java application code and the relational database management system (RDBMS).

This document guides you through setting up a Linux environment from scratch, writing a robust JDBC application that handles full CRUD (Create, Read, Update, Delete) architecture, compiling it via the terminal, and executing it cleanly.

1. Environment Architecture & Prerequisites

To follow this guide completely inside a Linux environment (Ubuntu/Debian or RHEL/CentOS), you must ensure that your system has the Java Development Kit (JDK) installed and a running database server. We will be using **MySQL / MariaDB** for this setup, which is standard across industry architectures.

Step 1.1: Install JDK and MySQL Server

Open your Linux terminal and run the following execution blocks to update your packages and install the modern OpenJDK environment along with the native database client/server utilities:

```
# Update your local package index
sudo apt update

# Install OpenJDK 17 Development Kit
sudo apt install openjdk-17-jdk -y

# Install MySQL Database Server
sudo apt install mysql-server -y
```

Step 1.2: Database Initialization & Table Configuration

Once installed, start and secure the MySQL engine. We must create a dedicated workspace database and a relational table where our Java code can issue modifications safely.

```
# Access the root terminal interface of MySQL
sudo mysql
```

Inside the interactive prompt interface of MySQL, execute the standard SQL queries below to build the table structure:

```
CREATE DATABASE company_db;
USE company_db;

CREATE TABLE employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
    salary DOUBLE NOT NULL
```

```
);
```

```
-- Establish an authorized database user account  
CREATE USER 'java_user'@'localhost' IDENTIFIED BY 'SecurePass123!';  
GRANT ALL PRIVILEGES ON company_db.* TO 'java_user'@'localhost';  
FLUSH PRIVILEGES;  
EXIT;
```

Senior Engineer Insight: Always isolate database privileges. Never embed the database root username and password directly inside application codeblocks or production configuration scripts.

2. Understanding JDBC Core Architecture

JDBC defines an abstraction layer utilizing four core interfaces that every enterprise developer must memorize:

- **DriverManager** : Manages the collection of database drivers and establishes network **Connection** instances via a target database URL.
- **Connection** : Represents a session with a specific database. Used to control transaction boundaries (commit/rollback).
- **PreparedStatement** : Precompiles SQL queries, prevents SQL Injection attacks, and handles dynamic value binding.
- **ResultSet** : A tabular data stream containing database matching records produced by executing queries.

3. Writing the Complete JDBC Application Code

We will construct an organized monolithic class file named `JdbcApplication.java`. This clean structure will present an interactive command-line interface (CLI) demonstrating full programmatic control over table records.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class JdbcApplication {
    // Database credentials and connectivity string configuration
    private static final String URL = "jdbc:mysql://localhost:3606/company_db";
    private static final String USER = "java_user";
    private static final String PASSWORD = "SecurePass123!";

    public static void main(String[] args) {
        // Automatically load the modern MySQL JDBC Driver class blueprint
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("Database Connector Driver not detected on Classpath!");
            e.printStackTrace();
            return;
        }

        try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
            Scanner scanner = new Scanner(System.in)) {

            System.out.println(">>> Connected successfully to Linux Enterprise DB Engine. <<<");
            while (true) {
                System.out.println("
--- JDBC DATABASE INTERACTION ENGINE ---");
                System.out.println("1. Display All Records (READ)");
                System.out.println("2. Insert New Record (CREATE)");
                System.out.println("3. Delete Target Record (DELETE)");
                System.out.println("4. Terminate Runtime Execution");
                System.out.print("Select Operational Vector [1-4]: ");

                int choice = scanner.nextInt();
                scanner.nextLine(); // Clear the buffer zone

                switch (choice) {
                    case 1 -> displayRecords(conn);
                    case 2 -> insertRecord(conn, scanner);
                    case 3 -> deleteRecord(conn, scanner);
                    case 4 -> {
                        System.out.println("Shutting down database context connections...");
                        return;
                    }
                    default -> System.out.println("Invalid operational parameter selected.");
                }
            }
        } catch (SQLException e) {
            System.err.println("Fatal Application Runtime Database Error encountered:");
            e.printStackTrace();
        }
    }

    private static void displayRecords(Connection conn) throws SQLException {
```

```

String query = "SELECT id, name, role, salary FROM employees";
try (Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(query)) {

    System.out.println("
===== EMPLOYEE DATABASE TABLE =====");
    System.out.printf("%-6s %-20s %-15s %-10s
", "ID", "NAME", "ROLE", "SALARY");
    System.out.println("-----");
    boolean containsData = false;
    while (rs.next()) {
        containsData = true;
        System.out.printf("%-6d %-20s %-15s $%-9.2f
",
                        rs.getInt("id"), rs.getString("name"), rs.getString("role"),
rs.getDouble("salary"));
    }
    if (!containsData) {
        System.out.println(" [No historical tracking records matching current query
context] ");
    }
}
}

```

```

private static void insertRecord(Connection conn, Scanner scanner) throws SQLException {
    String sql = "INSERT INTO employees (name, role, salary) VALUES (?, ?, ?)";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        System.out.print("Enter Employee Legal Name: ");
        String name = scanner.nextLine();
        System.out.print("Enter Enterprise Role Assignment: ");
        String role = scanner.nextLine();
        System.out.print("Enter Contractual Base Salary: ");
        double salary = scanner.nextDouble();

        pstmt.setString(1, name);
        pstmt.setString(2, role);
        pstmt.setDouble(3, salary);

        int rowsAffected = pstmt.executeUpdate();
        System.out.println(">>> Operation Executed: " + rowsAffected + " record successfully
committed.");
    }
}

private static void deleteRecord(Connection conn, Scanner scanner) throws SQLException {
    String sql = "DELETE FROM employees WHERE id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        System.out.print("Enter target Employee ID to eliminate from registry: ");
        int id = scanner.nextInt();

        pstmt.setInt(1, id);

        int rowsAffected = pstmt.executeUpdate();
        if (rowsAffected > 0) {
            System.out.println(">>> Operation Executed: Record removed from operational
state successfully.");
        } else {
            System.out.println(">>> Notification: Target index not found inside context
state.");
        }
    }
}
}

```

4. Compiling and Running via Linux Terminal

To run this program, you need the official MySQL JDBC Driver (a `.jar` file called Connector/J). Let's fetch it, structure our folders, and compile the application manually using the terminal command-line interface.

Step 4.1: Fetch the JDBC Driver Jar

Create a dedicated workspace directory, download the official connector archive via `wget`, and extract the internal JAR archive:

```

mkdir ~/jdbc_project && cd ~/jdbc_project
# Download the driver archive package
wget https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-8.2.0.tar.gz
# Extract the compressed archive
tar -xvf mysql-connector-j-8.2.0.tar.gz

```

This provides a driver file named `mysql-connector-j-8.2.0.jar` inside the extracted folder. Copy or reference this file to run your program.

Step 4.2: Compilation Flow and Dynamic Execution

Save the code from page 2 and 3 into a file named `JdbcApplication.java` inside your project directory. Compile it using the Java compiler (`javac`) and run it by specifying the classpath (`-cp`):

Phase	Command Execution Script
1. Compile	<code>javac JdbcApplication.java</code>
2. Execute	<code>java -cp .:mysql-connector-j-8.2.0/mysql-connector-j-8.2.0.jar JdbcApplication</code>

Note on Classpath Separators: In Linux operating environments, the classpath separation operator is a colon (`:`), whereas Windows environments use a semicolon (`;`).

Once you execute the launch command, you will see the operational runtime menu. Try inserting a few employee items and displaying them to observe standard JDBC persistence logic working natively on your Linux environment!